

网络空间安全实验基础

Hands-on Cybersecurity – Fundamentals

数据包嗅探和伪造

主讲人：张玉健



课程提纲

- 数据包的发送与接收
- 数据包嗅探
- 数据包伪造
- 工具：Python vs C
- 字节顺序与校验和

Sniffing

Spoofing

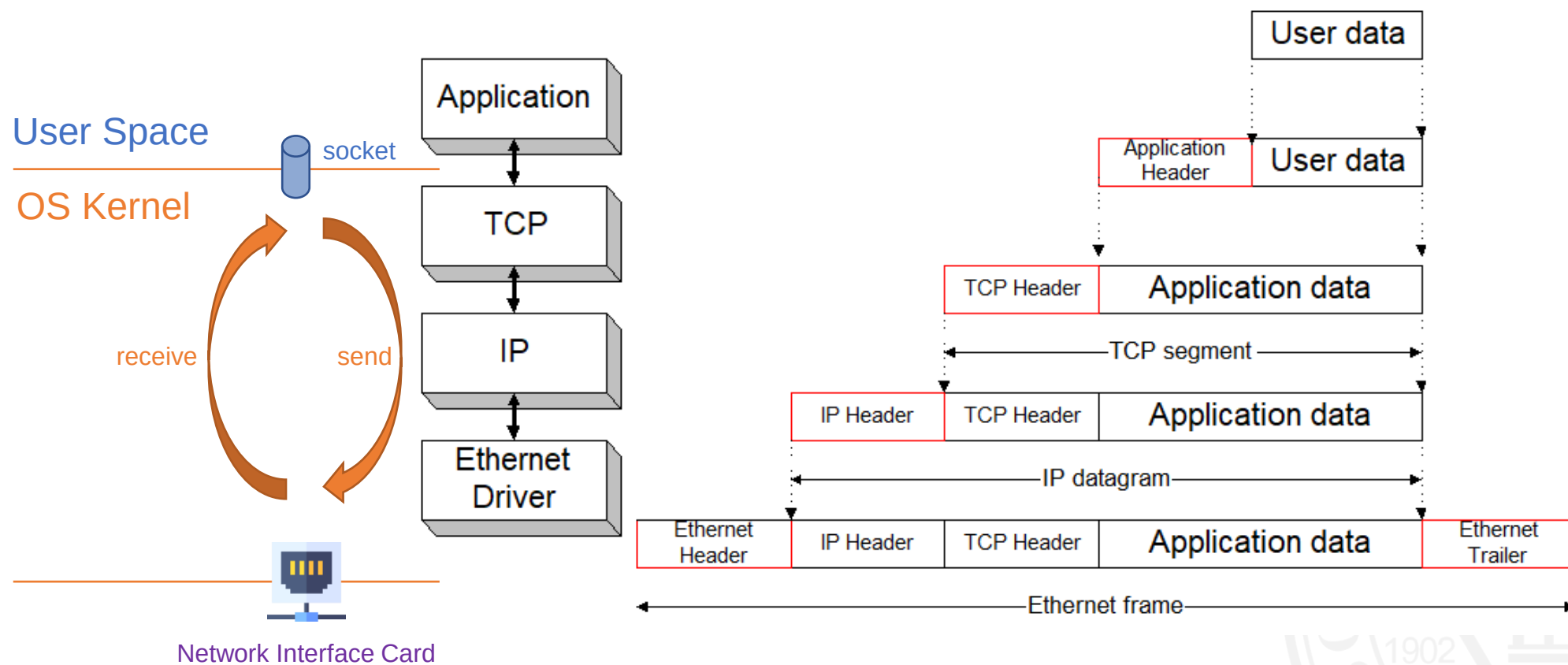
课程提纲

- 数据包的发送与接收
- 数据包嗅探
- 数据包伪造
- 工具：Python vs C
- 字节顺序与校验和

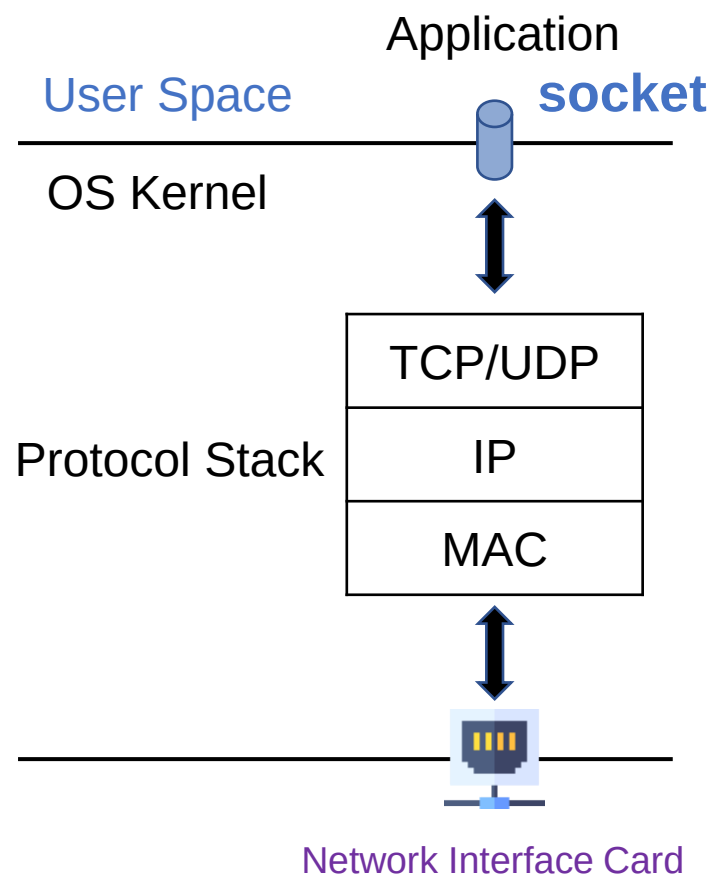
Sniffing

Spoofing

数据包的收发机制



Socket接口



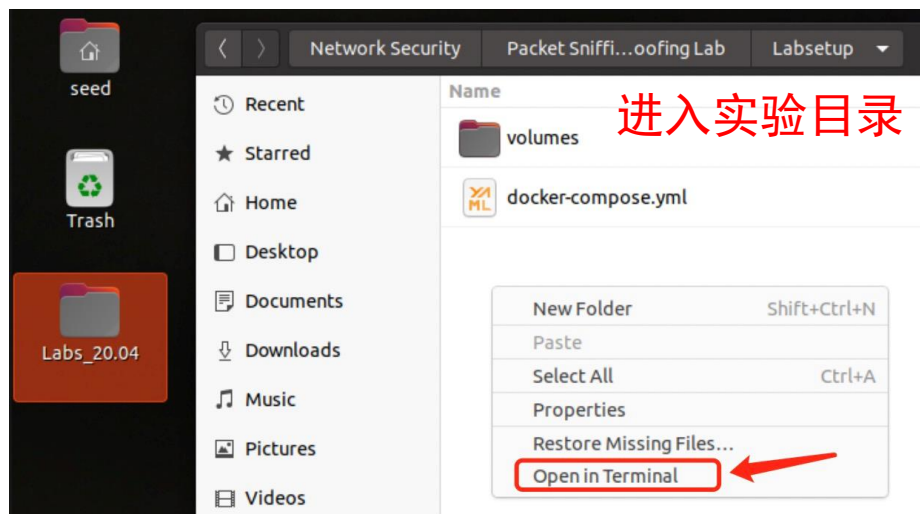
客户端

1. `gethostbyname()`
2. `socket()`
3. `connect()`
4. `write()/send()`
5. `read()/recv()`
6. `close()`

服务端

1. `socket()`
2. `bind()`
3. `listen()`
4. `while(whatever) {`
5. `accept()`
6. `read()/recv()`
7. `write()/send()`
8. `close()`
9. `}`
10. `close()`

实验环境



```
seed@VM: ~/.../Labsetup
[06/20/21]seed@VM:~/.../Labsetup$ ls
docker-compose.yml  volumes
[06/20/21]seed@VM:~/.../Labsetup$ dcbuild
attacker uses an image, skipping
host uses an image, skipping
[06/20/21]seed@VM:~/.../Labsetup$ dcup 启动容器
host-10.9.0.5 is up-to-date
seed-attacker is up-to-date
Attaching to host-10.9.0.5, seed-attacker

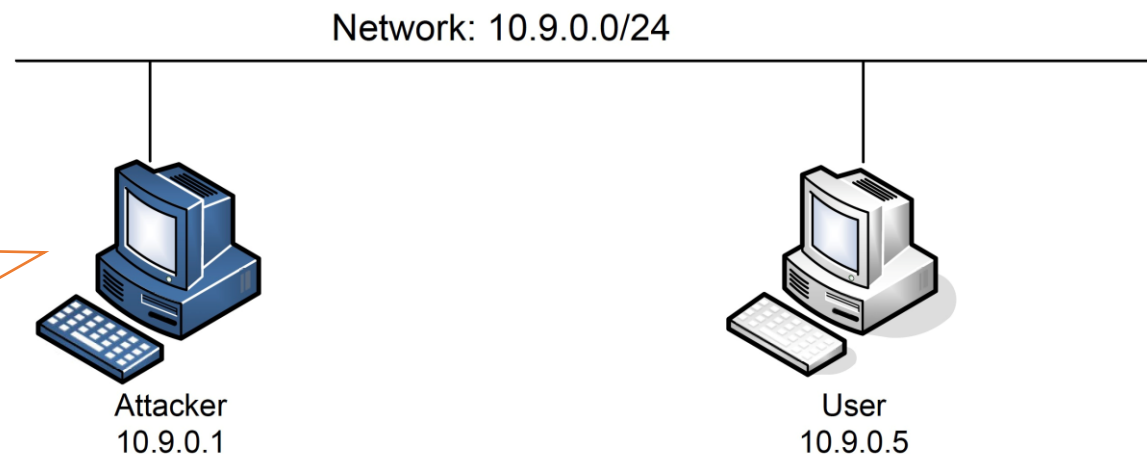
seed@VM: ~/.../Labsetup
[06/20/21]seed@VM:~/.../Labsetup$ dockps
54e7ebc15220  seed-attacker 10.9.0.1
91b8f203cdca  host-10.9.0.5 - 检查容器
```

挂载目录

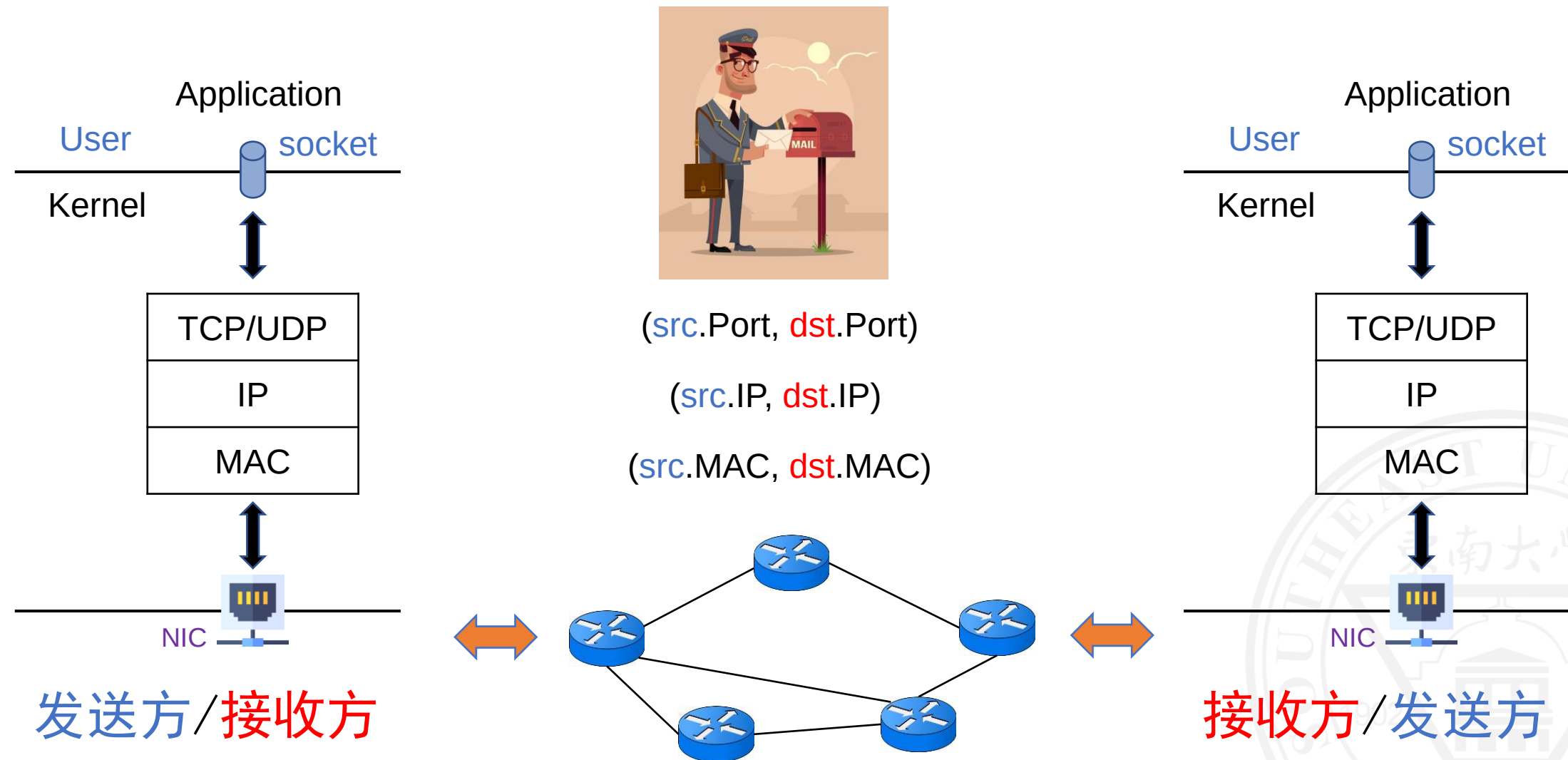
```
volumes:
- ./volumes:/volumes
```

网络模式

```
network_mode: host
```



数据包的发送/接收过程



思考：数据包传输的过程中哪些地址信息会改变，哪些不会？

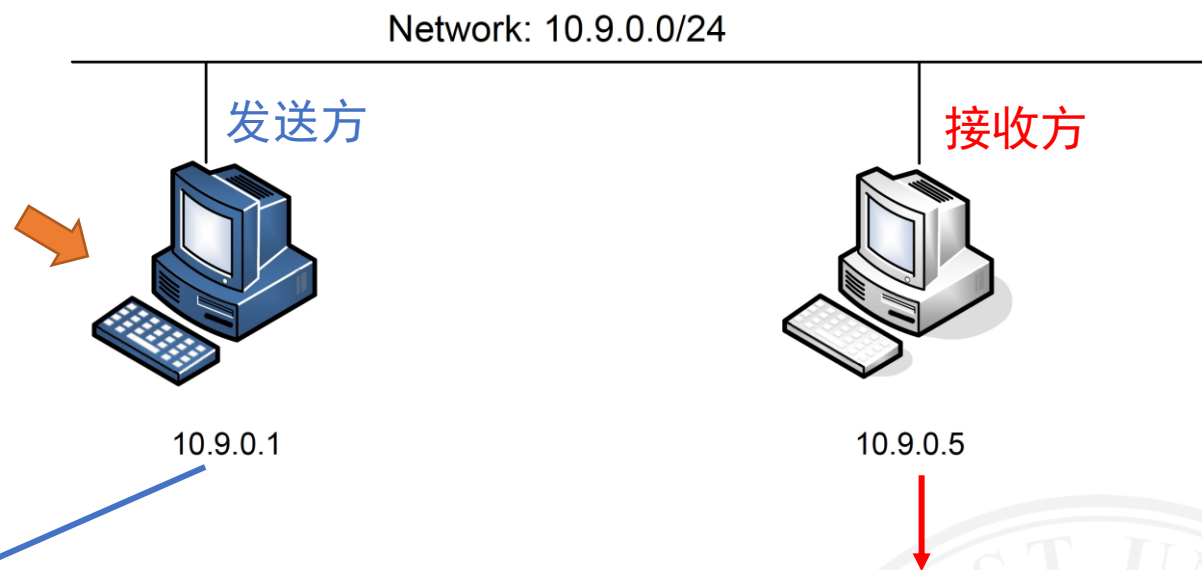
发送一个数据包 (UDP)



```
sender.py
1  #!/usr/bin/python3
2
3  import socket
4
5  IP = "10.9.0.5"
6  PORT = 9090
7  data = b'Hello World!'
8
9  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
10 sock.sendto(data, (IP, PORT))
```

IPv4

UDP



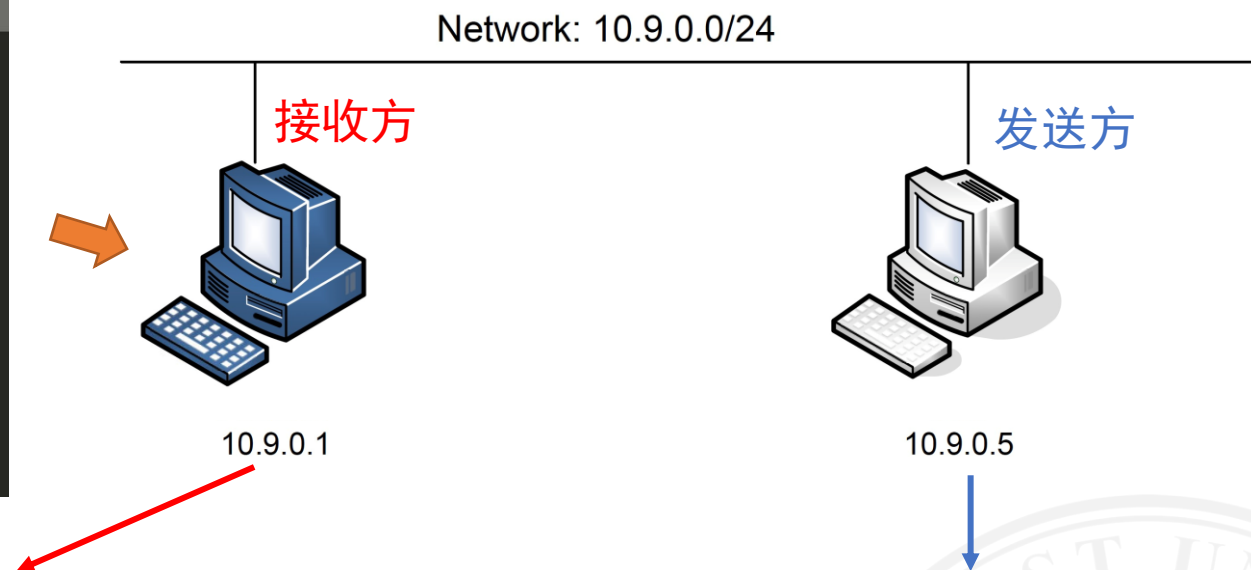
```
seed@VM: ~/.../volumes$ dockps
54e7ebc15220  seed-attacker
91b8f203cdca  host-10.9.0.5
[06/21/21]seed@VM:~/.../volumes$ docksh 54
root@VM:/# cd volumes/
root@VM:/volumes# sender.py
root@VM:/volumes#
```

```
seed@VM: ~/.../volumes$ docksh 91
root@91b8f203cdca:/# nc -luv 9090
Bound on 0.0.0.0 9090
Connection received on 10.9.0.1 55102
Hello World!
```

接收一个数据包 (UDP)



```
receiver.py
1  #!/usr/bin/python3
2
3  import socket
4
5  IP = "0.0.0.0"
6  PORT = 9090
7
8  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
9  sock.bind((IP, PORT))
10
11 while True:
12     data, (ip, port) = sock.recvfrom(1024)
13     print("Sender: {} and Port: {}".format(ip, port))
14     print("Received message: {}".format(data))
```



```
seed@VM: ~/.../volumes
[06/21/21] seed@VM: ~/.../volumes$ dockps
54e7ebc15220  seed-attacker
91b8f203cdca  host-10.9.0.5
[06/21/21] seed@VM: ~/.../volumes$ docksh 54
root@VM:/# cd volumes/
root@VM:/volumes# receiver.py
Sender: 10.9.0.5 and Port: 56125
Received message: b'hello\n'
```

```
seed@VM: ~
[06/21/21] seed@VM: ~$ docksh 91
root@91b8f203cdca:/# nc -u 10.9.0.1 9090
hello
```

课程提纲

- 数据包的发送与接收
- 数据包嗅探
- 数据包伪造
- 工具：Python vs C
- 字节顺序与校验和

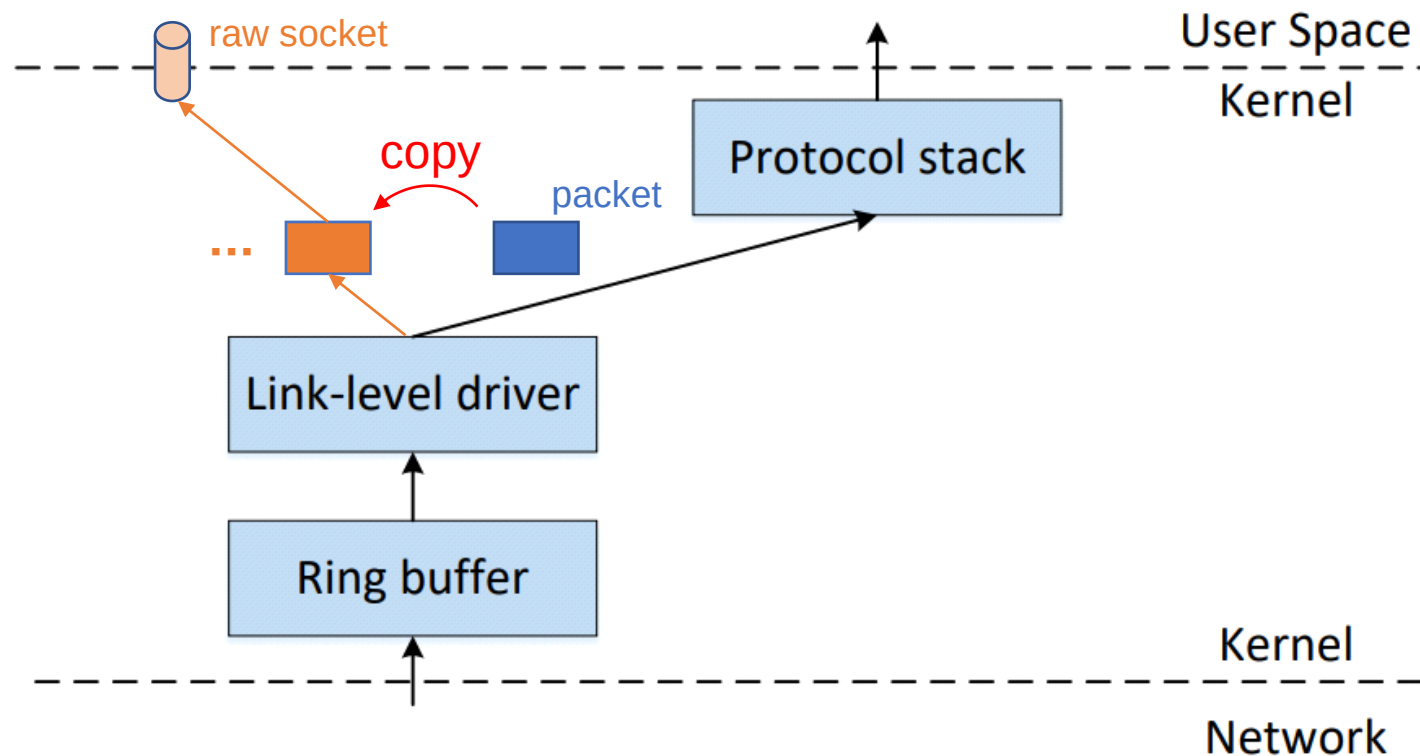
Sniffing

Spoofing

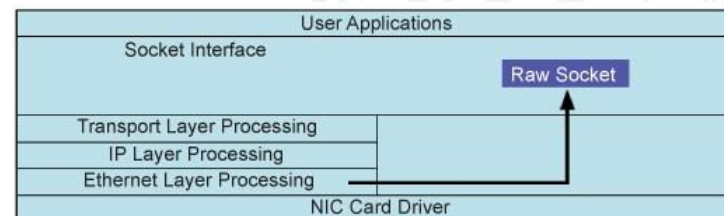
获得数据包的拷贝

➤ 原始套接字 (Raw Socket)

- 功能：绕过系统内核的协议栈，发送/接收原始的数据包
- 条件：网卡支持并开启混杂模式 (Promiscuous Mode)



```
int main() {  
    int PACKET_LEN = 512;  
    char buffer[PACKET_LEN];  
    struct sockaddr saddr;  
    struct packet_mreq mr;  
  
    // Create the raw socket  
    int sock = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));  
  
    // Turn on the promiscuous mode.  
    mr.mr_type = PACKET_MR_PROMISC;  
    setsockopt(sock, SOL_PACKET, PACKET_ADD_MEMBERSHIP, &mr, sizeof(mr));  
  
    // Getting captured packets  
    while (1) {  
        int data_size = recvfrom(sock, buffer, PACKET_LEN, 0,  
                                &saddr, (socklen_t*)sizeof(saddr));  
        if (data_size) printf("Got one packet\n");  
    }  
  
    close(sock);  
    return 0;  
}
```

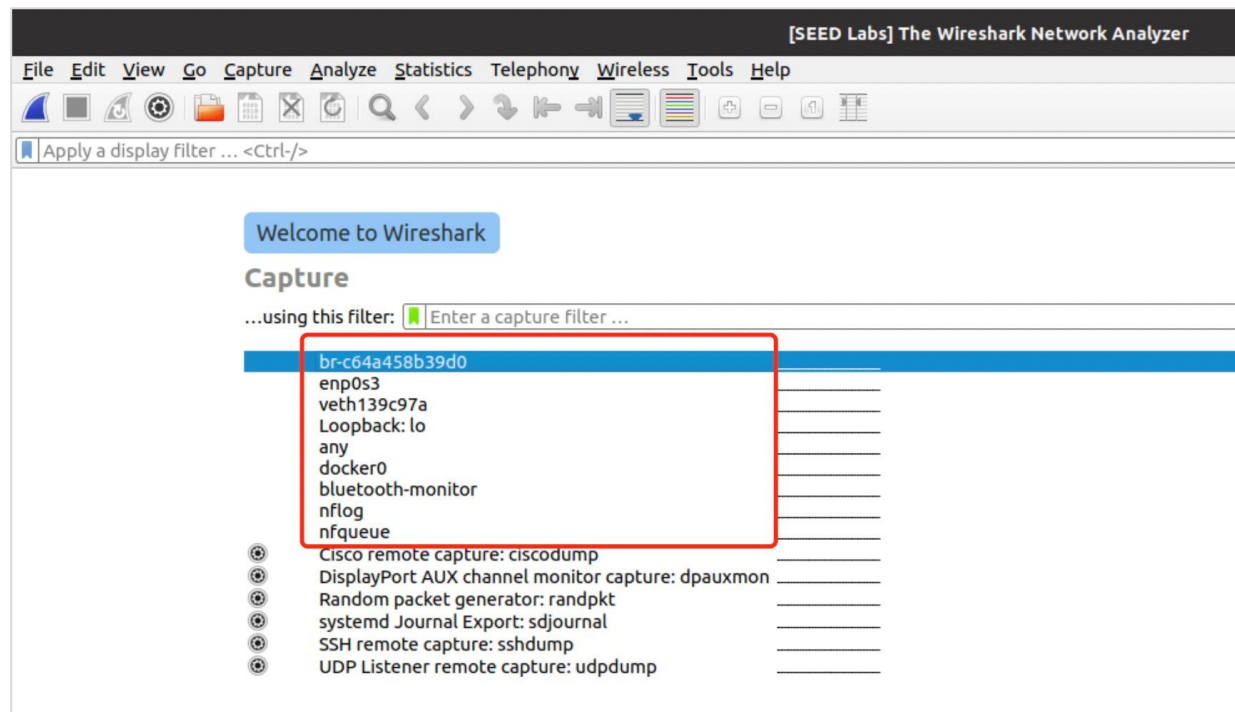


嗅探数据包的工具

➤ 抓包工具

- **基础**: PCAP (Packet Capture API)
- **例如**: Wireshark, tcpdump, Scapy, McAfee, Nmap, Snort, ...

```
seed@VM: ~  
[06/21/21]seed@VM:~$ ifconfig  
br-c64a458b39d0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 10.9.0.1 netmask 255.255.255.0 broadcast 10.9.0.255  
    inet6 fe80::42:a2ff:fedc:59d7 prefixlen 64 scopeid 0x20<link>  
    ether 02:42:a2:dc:59:d7 txqueuelen 0 (Ethernet)  
    RX packets 108 bytes 6341 (6.3 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 223 bytes 20605 (20.6 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500  
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255  
    ether 02:42:4d:f3:41:38 txqueuelen 0 (Ethernet)  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 0 bytes 0 (0.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 10.0.2.11 netmask 255.255.255.0 broadcast 10.0.2.255  
    inet6 fe80::e695:929f:5fbc:8fd7 prefixlen 64 scopeid 0x20<link>  
    ether 08:00:27:d6:9e:52 txqueuelen 1000 (Ethernet)  
    RX packets 2546 bytes 493545 (493.5 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 3090 bytes 324049 (324.0 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0
```



使用工具嗅探数据包



人为产生一些流量（例如访问网页），抓取**本地网卡**的数据包，并进行简单分析

[SEED Labs] **enp0s3**

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	2021-06-21 03:4...	10.0.2.11	8.8.8.8	DNS	95	Standard query 0xd2bf A detectportal.firefox.com OPT
2	2021-06-21 03:4...	10.0.2.11	8.8.8.8	DNS	95	Standard query 0xe7b6 AAAA detectportal.firefox.com OPT
3	2021-06-21 03:4...	10.0.2.11	8.8.8.8	DNS	91	Standard query 0x4471 A seedsecuritylabs.org OPT
4	2021-06-21 03:4...	10.0.2.11	8.8.8.8	DNS	91	Standard query 0x4eef AAAA seedsecuritylabs.org OPT
5	2021-06-21 03:4...	8.8.8.8	10.0.2.11	DNS	206	Standard query response 0xd2bf A detectportal.firefox.com CNA...
6	2021-06-21 03:4...	10.0.2.11	8.8.8.8	DNS	100	Standard query 0x5ce8 A location.services.mozilla.com OPT
7	2021-06-21 03:4...	10.0.2.11	8.8.8.8	DNS	100	Standard query 0x7975 AAAA location.services.mozilla.com OPT
8	2021-06-21 03:4...	10.0.2.11	8.8.8.8	DNS	106	Standard query 0x276c A content-signature-2.cdn.mozilla.net 0...
9	2021-06-21 03:4...	10.0.2.11	8.8.8.8	DNS	106	Standard query response 0xcf2c AAAA content-signature-2.cdn.mozilla.ne...
10	2021-06-21 03:4...	8.8.8.8	10.0.2.11	DNS	155	Standard query response 0x4471 A seedsecuritylabs.org A 185.1...
11	2021-06-21 03:4...	8.8.8.8	10.0.2.11	DNS	162	Standard query response 0x4eef AAAA seedsecuritylabs.org SOA ...
12	2021-06-21 03:4...	8.8.8.8	10.0.2.11	DNS	248	Standard query response 0x5ce8 A location.services.mozilla.co...
13	2021-06-21 03:4...	10.0.2.11	185.199.108.153	TCP	74	40400 → 443 [SYN] Seq=1042784536 Win=64240 Len=0 MSS=1460 SAC...
14	2021-06-21 03:4...	8.8.8.8	10.0.2.11	DNS	234	Standard query response 0x7975 AAAA location.services.mozilla...
15	2021-06-21 03:4...	10.0.2.11	8.8.8.8	DNS	109	Standard query 0xd1e8 AAAA locprod2-elb-us-west-2.prod.mozaws...
16	2021-06-21 03:4...	8.8.8.8	10.0.2.11	DNS	370	Standard query response 0xcf2c AAAA content-signature-2.cdn.m...
17	2021-06-21 03:4...	8.8.8.8	10.0.2.11	DNS	210	Standard query response 0x276c A content-signature-2.cdn.moz...
18	2021-06-21 03:4...	10.0.2.11	99.84.138.38	TCP	74	55200 → 443 [SYN] Seq=868841659 Win=64240 Len=0 MSS=1460 SACK...

Frame 1: 95 bytes on wire (760 bits), 95 bytes captured (760 bits) on interface enp0s3, id 0

Ethernet II, Src: PcsCompu_d6:9e:52 (08:00:27:d6:9e:52), Dst: RealtekU_12:35:00 (52:54:00:12:35:00)

Internet Protocol Version 4, Src: 10.0.2.11, Dst: 8.8.8.8

User Datagram Protocol, Src Port: 39935, Dst Port: 53

Domain Name System (query)

```
0000 52 54 00 12 35 00 08 00 27 d6 9e 52 08 00 45 00 RT..5...'.R..E.
0010 00 51 6a 07 40 00 40 11 b4 7a 0a 00 02 0b 08 08 .Qj.@.@.z.....
0020 08 08 9b ff 00 35 00 3d 1c 69 d2 bf 01 00 00 01 .....5.=.i.....
0030 00 00 00 00 00 01 0c 64 65 74 65 63 74 70 6f 72 .....d etectpor
0040 74 61 6c 07 66 69 72 65 66 6f 78 03 63 6f 6d 00 tal.fire fox.com.
0050 00 01 00 01 00 00 29 02 00 00 00 00 00 00 00 00 .....).
```

使用Python+Scapy实现数据包处理

➤ Scapy

- **作用**：Python 网络协议分析库，支持对网络协议的分析、生成和发送

功能	代码示例
导入 Scapy 库	<code>from scapy.all import *</code>
捕获网络数据包	<code>packets = sniff(count=10)</code>
分析数据包	<code>for packet in packets: print(packet.show())</code>
发送数据包	<code>packet = IP(dst="www.google.com")/ICMP() send(packet)</code>



编程实现数据包嗅探（实验任务一.a）



➤ 数据包嗅探

• 实验步骤

- ① 编写sniffer.py程序，保存至VM任意文件夹（建议选择之前的volumes目录）
 - ② 在VM上添加上述程序文件的执行权限
 - ③ 以root身份（用sudo命令）启动sniffer.py程序，产生相关流量并观察现象
- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

```
sniffer.py
1  #!/usr/bin/python3
2
3  from scapy.all import *
4
5  def print_pkt(pkt):
6      pkt.show()
7
8  pkt = sniff(iface='enp0s3', filter='icmp', prn=print_pkt)
```

思考：(1) 如何指定网络接口？

(2) 如何过滤报文？

(3) 为什么一定需要root权限？

```
// Make the program executable
# chmod a+x sniffer.py

// Run the program with the root privilege
# sniffer.py

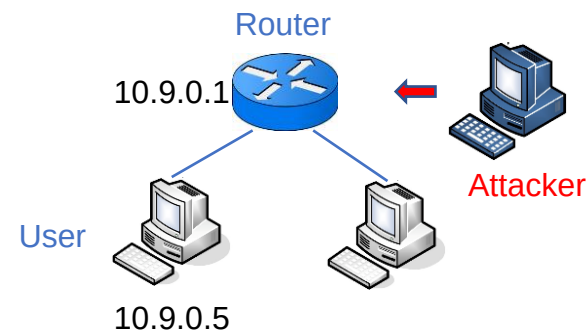
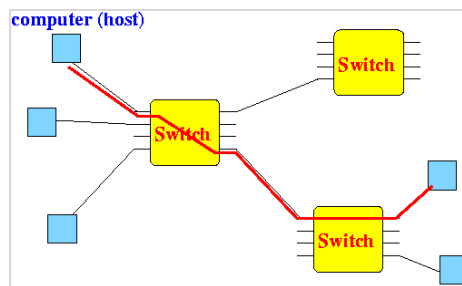
// Switch to the "seed" account, and
// run the program without the root privilege
# su seed
$ sniffer.py
```

嗅探其他机器的数据包（实验任务一.b）



➤ 嗅探的条件

- **问题：**什么样的数据包会到达当前网络接口？



➤ 数据包嗅探

- 实验步骤
 - ① 修改sniffer.py程序，选择合适的网络接口
 - ② 在Attacker容器上启动程序
 - ③ 在User容器上产生相关流量并观察现象
- **要求：**记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。



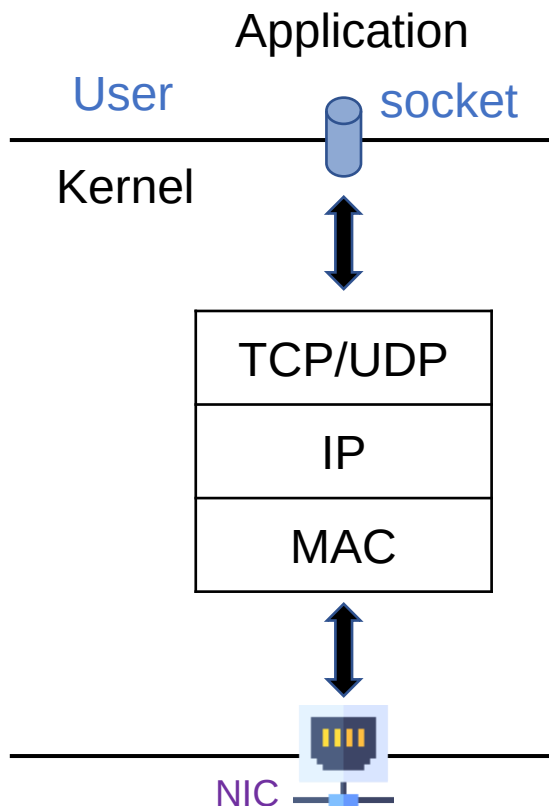
课程提纲

- 数据包的发送与接收
- 数据包嗅探
- 数据包伪造
- 工具：Python vs C
- 字节顺序与校验和

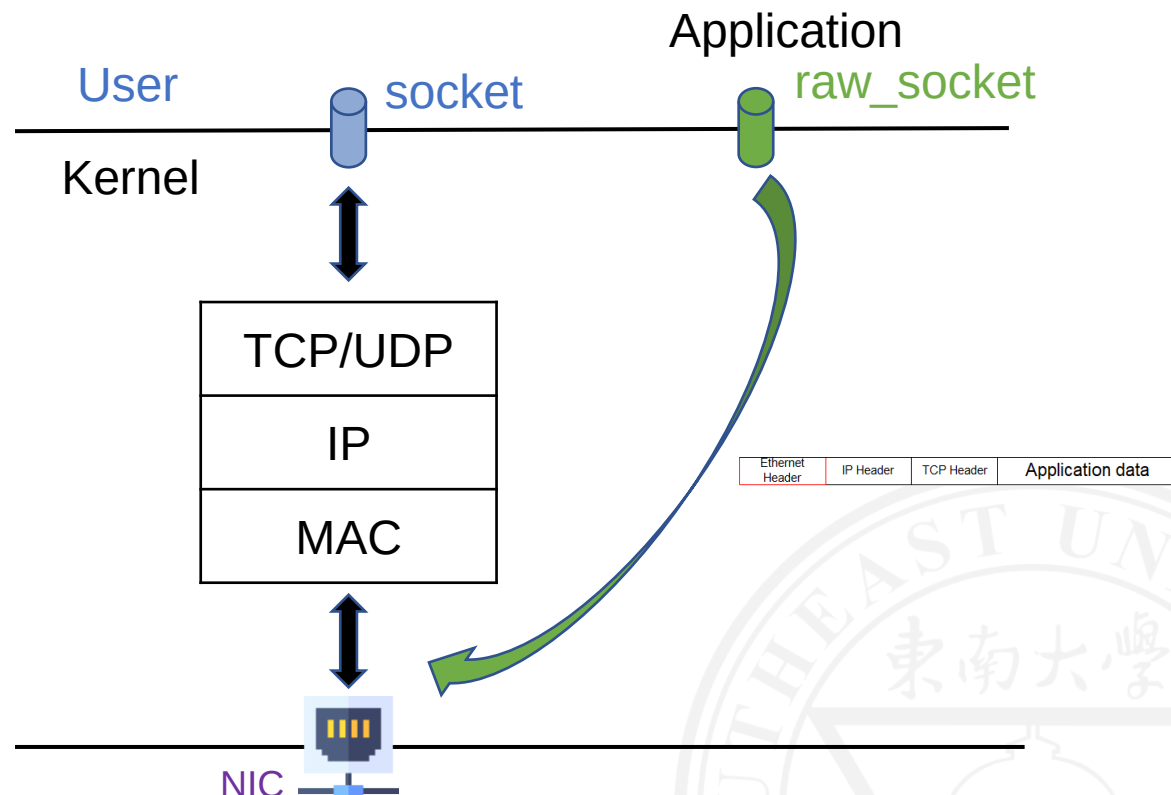
Sniffing

Spoofing

通过RAW_SOCKET发送原始数据包



正常socket



raw_socket

编程实现数据包伪造（实验任务二）



➤ 数据包伪造

• 实验步骤

- ① 编写spoofer.py程序，保存至volumes目
- ② 在Attacker容器上添加上述程序文件的执行权限
- ③ 打开wireshark选好相关网络接口，启动spoofer.py程序，观察现象

• 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

```
spoofer.py
1  #!/usr/bin/python3
2
3  from scapy.all import *
4
5  a = IP()
6  a.src = '10.9.0.3'
7  a.dst = '10.9.0.99'
8  b = ICMP(type=0)
9  p = a/b
10 send(p)
```

思考：(1) 不填写源地址会发生什么？为什么？
(2) 目标地址不在当前网段会发生什么？为什么？

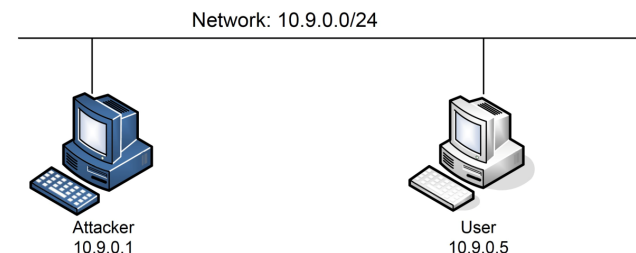
嗅探并伪造（实验任务三）



➤ 嗅探并伪造ICMP报文

- 实验目标：受害者（10.9.0.5）ping任意IP地址都会得到响应
- 实验步骤

- ① 编写sniff_spoof_icmp.py程序，保存至volumes目录
- ② 在Attacker容器上添加上述程序文件的执行权限，并启动sniff_spoof_icmp.py
- ③ 在User容器上执行下列ping命令，观察现象并进行分析



```
ping 1.2.3.4      # a non-existing host on the Internet
ping 10.9.0.99    # a non-existing host on the LAN
ping 8.8.8.8      # an existing host on the Internet
```

不会成功，为什么？

- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

课程提纲

- 数据包的发送与接收
- 数据包嗅探
- 数据包伪造
- 工具：Python vs C
- 字节顺序与校验和

Sniffing

Spoofing

Python+Scapy

```
#!/usr/bin/python3

from scapy.all import *

pkt = sniff(iface='enp0s3',
            filter='icmp or udp',
            count=10)

pkt.summary()
```

```
#!/usr/bin/python3

from scapy.all import *

def process_packet(pkt):
    #hexdump(pkt)
    pkt.show()
    print("-----")

f = 'udp and dst portrange 50-55 or icmp'

sniff(iface='enp0s3', filter = f, prn=process_packet)
```

```
>>> hexdump(pkt)
0000  52 54 00 12 35 00 08 00 27 77 2E C3 08 00 45 00  RT..5...'w....E.
0010  00 54 F2 29 40 00 40 01 2C 68 0A 00 02 08 08 08  .T.)@.@.,h.....
0020  08 08 08 00 98 01 10 C7 00 02 B8 66 65 5E 3A 6D  .....fe^:m
0030  0C 00 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15  .....
0040  16 17 18 19 1A 1B 1C 1D 1E 1F 20 21 22 23 24 25  .....! "$%
0050  26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35  &'()*+,-./012345
0060  36 37                                           67
```

```
>>> pkt.show()
###[ Ethernet ]###
  dst      = 52:54:00:12:35:00
  src      = 08:00:27:77:2e:c3
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  ...
  proto    = icmp
  chksum   = 0x3c9a
  src      = 10.0.2.8
  dst      = 8.8.8.8
  \options \
###[ ICMP ]###
  type     = echo-request
  code     = 0
  chksum   = 0x6905
  id       = 0x107a
  seq      = 0x2
###[ Raw ]###
  load     = '\x90ee^\x91\xb7\ ...'
```

用C编写嗅探程序（实验任务四）



➤ 嗅探ICMP数据包

- 实验目标：了解C语言实现相同功能的方法和底层实现
- 实验步骤
 - ① 编写sniffer.c程序，保存至volumes目录，编译成可执行文件
 - ② 在Attacker容器上添加上述程序文件的执行权限
 - ③ 在User容器上发起ICMP或UDP数据包，观察现象并进行分析
- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

```
int main()
{
    pcap_t *handle;
    char errbuf[PCAP_ERRBUF_SIZE];
    struct bpf_program fp;
    char filter_exp[] = "udp or icmp";
    bpf_u_int32 net;

    // Step 1: Open live pcap session on NIC with interface name
    handle = pcap_open_live("enp0s3", 8192, 1, 1000, errbuf);

    // Step 2: Compile filter_exp into BPF psuedo-code
    pcap_compile(handle, &fp, filter_exp, 0, net);
    if (pcap_setfilter(handle, &fp) != 0) {
        pcap_perror(handle, "Error:");
        exit(EXIT_FAILURE);
    }

    // Step 3: Capture packets
    pcap_loop(handle, -1, got_packet, NULL);

    pcap_close(handle); //Close the handle
    return 0;
}
```

```
void got_packet(u_char *args, const struct pcap_pkthdr *header,
                const u_char *packet)
{
    printf("Got a packet\n");
    struct ethheader *eth=(struct ethheader *)packet;

    if(ntohs(eth->ether_type) == 0x800) {
        struct ipheader *ip = (struct ipheader *) (packet +
            sizeof(struct ethheader));
        printf("    From: %s\n", inet_ntoa(ip->iph_sourceip));
        printf("    To: %s\n", inet_ntoa(ip->iph_destip));

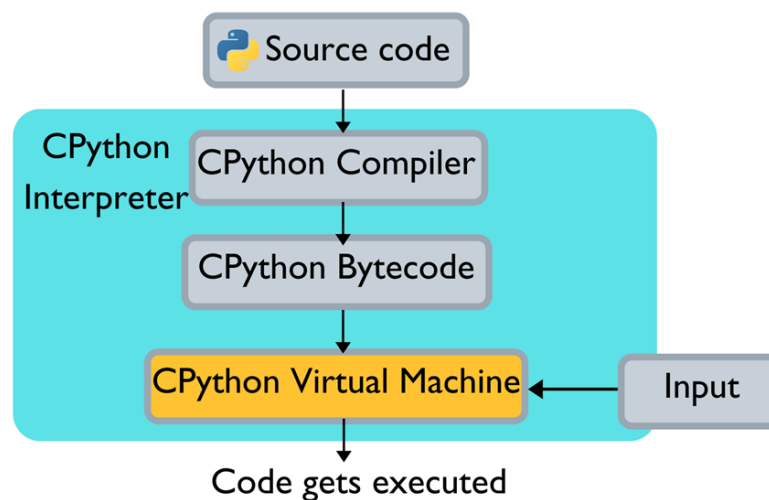
        switch(ip->iph_protocol) {
            case IPPROTO_TCP:
                printf("    Protocol: TCP\n");
                break;
            case IPPROTO_UDP:
                printf("    Protocol: UDP\n");
                break;
            case IPPROTO_ICMP:
                printf("    Protocol: ICMP\n");
                break;
            default:
                printf("    Protocol: Others\n");
                break;
        }
    }
}
```

Python vs C

两种编程语言比较

	Python	C
编程复杂度	容易	复杂
执行效率	低	高

Scapy: 106 数据包/秒
C: 4000 数据包/秒



混合法

```
#!/usr/bin/python3
from scapy.all import *
from socket import AF_INET, SOCK_DGRAM, socket

sock = socket(AF_INET, SOCK_DGRAM)
sock.bind(('0.0.0.0', 1053))

while True:
    request, addr = sock.recvfrom(4096)
    DNSreq = DNS(request)
    query = DNSreq.qd.qname
    print(query.decode('ascii'))

    Anssec = DNSRR(rrname=DNSreq.qd.qname, type='A',
                   rdata='1.1.1.1', ttl=259200)
    NSsec1 = DNSRR(rrname="example.com", type='NS',
                   rdata='ns1.example.com', ttl=259200)
    NSsec2 = DNSRR(rrname="example.com", type='NS',
                   rdata='ns2.example.com', ttl=259200)
    Addsec1 = DNSRR(rrname='ns1.example.com', type='A',
                    rdata='1.2.3.4', ttl=259200)
    Addsec2 = DNSRR(rrname='ns2.example.com', type='A',
                    rdata='1.2.3.5', ttl=259200)
    DNSpkt = DNS(id=DNSreq.id, aa=1, rd=0, qr=1,
                 qdcount=1, ancount=1, nscount=2, arcount=2,
                 qd=DNSreq.qd, an=Anssec,
                 ns=NSsec1/NSsec2, ar=Addsec1/Addsec2)
    print(repr(DNSpkt))
    sock.sendto(bytes(DNSpkt), addr)
```



```
# Construct the IP and UDP header
IPpkt = IP(dst=dstIP, src=srcIP)
UDPkpt = UDP(dport=33333, sport=53, chksum=0)

# Construct the DNS header and records
Qdsec = DNSQR(qname=targetName)
Anssec = DNSRR(rrname=targetName, type='A', rdata='1.1.1.1', ttl=259200)
NSsec = DNSRR(rrname=targetDomain, type='NS', rdata=attackerNS, ttl=259200)
DNSpkt = DNS(id=0xAAAA, aa=1, rd=1, qr=1,
             qdcount=1, ancount=1, nscount=1, arcount=0,
             qd=Qdsec, an=Anssec, ns=NSsec)

# Stack all the layers together and save the result to a file.
Replypkt = IPpkt/UDPkpt/DNSpkt
with open('ip_resp.bin', 'wb') as f:
    f.write(bytes(Replypkt))
```

Python构造报文

```
void send_dns_response(unsigned char *ip_resp, int n_resp,
                      unsigned char *src_ip, char * name,
                      unsigned short transaction_id)
{
    // Modify the src IP in the IP header (offset=12)
    int ip = (int) inet_addr(src_ip);
    memcpy(ip_resp+12, (void *) &ip, 4);

    // Modify the name in the question field (offset=41)
    memcpy(ip_resp+41, name, 5);

    // Modify the name in the answer field (offset=64)
    memcpy(ip_resp+64, name, 5);

    // Modify the transaction ID field (offset=28)
    unsigned short id[2];
    *id = htons(transaction_id);
    memcpy(ip_resp+28, (void *) id, 2);

    // Send the IP packet out
    send_raw_packet(ip_resp, n_resp);
}
```

C发送报文

课程提纲

- 数据包的发送与接收
- 数据包嗅探
- 数据包伪造
- 工具：Python vs C
- 字节顺序与校验和

Sniffing

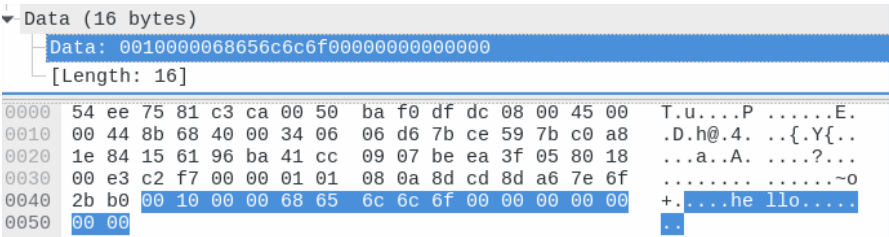
Spoofing

字节顺序

int value = 0x11223344

0x1003	44
0x1002	33
0x1001	22
0x1000	11

0x1003	11
0x1002	22
0x1001	33
0x1000	44



小端（大部分主机字节序） 大端（网络字节序）

字节序转换

宏函数	描述
htons()	无符号短整数从本机字节序转成网络字节序
htonl()	无符号整数从本机字节序转成网络字节序
ntohs()	无符号短整数从网络字节序转成本机字节序
ntohl()	无符号整数从网络字节序转成本机字节序

校验和 (Checksum)

➤ 校验和

- 原理：将数据流视为16位整数流进行重复叠加计算（RFC1071）
- 作用：保证数据包的完整性和正确性
- 注意：普通socket编程时，内核会自动计算；Raw Socket需用户自己关注（Python提供了便利）

```
unsigned short csum(unsigned char *addr, int count)
{
    /* Compute Internet Checksum for "count" bytes
     *   beginning at location "addr".
     */
    register long sum = 0;
    while( count > 1 ) {
        /* This is the inner loop */
        sum += (unsigned short) addr++;
        count -= 2;
    }
    /* Add left-over byte, if any */
    if( count > 0 )
        sum += (unsigned char *) addr;
    /* Fold 32-bit sum to 16 bits */
    while (sum >> 16)
        sum = (sum & 0xffff) + (sum >> 16);
    return ~sum;
}
```

```
seed@10.0.2.6:~$ python3
Python 3.5.2 (default, Nov 17 2016, 17:05:23)
[GCC 5.4.0 20160609] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from scapy.all import *
>>> ls(IP)
version      : BitField (4 bits)           = (4)
ihl          : BitField (4 bits)           = (None)
tos          : XByteField                  = (0)
len          : ShortField                  = (None)
id           : ShortField                  = (1)
flags        : FlagsField (3 bits)         = (<Flag 0 (>))
frag         : BitField (13 bits)          = (0)
ttl          : ByteField                   = (64)
proto        : ByteEnumField               = (0)
chksum       : XShortField                  = (None)
src          : SourceIPField               = (None)
dst          : DestIPField                 = (None)
options      : PacketListField             = ([])
```

数据包发送前将该字段置0，Python会帮助重新计算

小结

※ 数据包嗅探

- 数据包发送和接收的原理
- 嗅探其他机器数据包的条件

※ 数据包伪造

- 使用Scapy伪（构）造数据包
- 字节顺序和校验和

※ Python vs C

